

Qwen's next architecture appears in a preview: half the hidden size, 512 experts, 62% less KV cache.

Qwen3.8-Flash-Next ships a new model_type, qwen4_exp: a 512-expert MoE with 10 active per token, replacing a dense FFN.

MODELS & RELEASES

QWEN

EXAMPLE — CI VERIFICATION PENDING

THE DIFF

- hidden 5120, 64 layers, 4 KV heads, dense FFN (Qwen3.8-27B)
- + hidden 2560, 48 layers, 2 KV heads, 512-expert MoE (Flash-Next)

WHY IT MATTERS
Fewer KV heads and 12 full-attention layers change what one GPU can hold, before any kernel work.

WHO SHOULD CARE
Anyone sizing GPUs for long-context serving, or comparing this preview against Qwen3.8-27B.

THE NUMBER

24 KiB

KV cache per token, against 64 KiB for Qwen3.8-27B — 62.5% less

- Only full-attention layers grow a cache: 12 of 48 layers here, 16 of 64 before.
- 12 layers × 2 tensors × 2 KV heads × 256 head dim × 2 bytes (bf16) = 24 KiB.
- The other 36 layers are linear attention: a fixed-size state instead — see item 02.
- 512 experts with 10 active means 2.0% of expert FFN weights touch a given token.

RUN IT — 13 lines, CPU, offline, < 1 s

```
import json, pathlib
H = pathlib.Path(__file__).parent
cfg = lambda n: json.loads((H / n).read_text())["text_config"]
old, new = cfg("config_qwen38_27b.json"), cfg("config_flash_next.json")

def kv_per_token(c):
    # only full-attention layers hold a growing cache
    full = c["layer_types"].count("full_attention")
    return full * 2 * c["num_key_value_heads"] * c["head_dim"] * 2 # K+V, bfloat16

for name, c in (("Qwen3.8-27B", old), ("Qwen3.8-Flash-Next", new)):
    print(f"{name:20} layers={c['num_hidden_layers']:3} full_attn="
          f"{c['layer_types'].count('full_attention'):3} kv_heads={c['num_key_value_heads']}"
          f" kv/token={kv_per_token(c)//1024:3} KiB")

print(f"KV cache per token {100*(1-kv_per_token(new)/kv_per_token(old)):.1f}% smaller")
print(f"experts {new['num_experts']} total, {new['num_experts_per_tok']} active "
      f"{100*new['num_experts_per_tok']/new['num_experts']:.1f}% per token")
```

Qwen3.8-27B	layers= 64	full_attn= 16	kv_heads=4	kv/token= 64 KiB
Qwen3.8-Flash-Next	layers= 48	full_attn= 12	kv_heads=2	kv/token= 24 KiB
KV cache per token	62.5% smaller			
experts	512 total, 10 active (2.0% per token)			

WATCH OUT

CAVEAT
This is a preview checkpoint with an experimental model_type, and the two models are different sizes. The diff shows a direction of travel, not a like-for-like benchmark.

SOURCES & LINKS — every claim above traces to one of these

TYPE	WHAT	URL
Primary source	Original announcement / docs	huggingface.co/Qwen/Qwen3.8-Flash-Next
Runnable example	This exact script, re-run by CI	github.com/marccocm28/ai-daily-diff/tree/main/examples/2026-09-01-qwen-flash-next-config
Slides, video, brief	Everything from this episode	marccocm28.github.io/ai-daily-diff/2026-09-01/

Those linear-attention layers keep a fixed state in FP32. Quantizing it frees 69% of that memory.

DAMP quantizes recurrent states to 9.9 bits per value on average, keeping high-error channels precise instead of quantizing uniformly.

COST & LIMITS

ARXIV

EXAMPLE — CI VERIFICATION PENDING

THE DIFF

- recurrent state stored in FP32 – 32 bits per value
- + critical channels kept high, the rest INT8 – 9.9 bits average

WHY IT MATTERS

Uniform INT8 already hurts reasoning accuracy and INT4 destroys it. Mixed precision is what makes the saving usable.

WHO SHOULD CARE

Anyone serving Gated DeltaNet or Kimi Delta Attention models at batch, where state memory competes with weights.

THE NUMBER

6.75 →
2.09 GiB

recurrent-state memory at batch 64, computed from the Flash-Next config

- $1 - 9.9/32 = 69.06\%$: the paper's headline 69.1% follows from its own bit budget.
- 36 linear-attention layers × 48 value heads × 128 × 128 = 28,311,552 values per sequence.
- Also reported: up to 2.01× faster state-update kernel, up to 10.9% lower full-model TPOT.
- Uniform quantization fails here: INT8/FP8 degrade reasoning, INT4/NVFP4 collapse it.

RUN IT — 11 lines, CPU, offline, < 1 s

```
import json, pathlib
c = json.loads((pathlib.Path(__file__).parent / "config_flash_next.json").read_text())["text_config"]

linear = c["layer_types"].count("linear_attention") # layers with a fixed-size state
per_layer = c["linear_num_value_heads"] * c["linear_key_head_dim"] * c["linear_value_head_dim"]
values = linear * per_layer # state values per sequence
BATCH = 64

for label, bits in (("FP32, as stored today", 32.0), ("DAMP, 9.9 bit/value", 9.9)):
    per_seq = values * bits / 8
    print(f"{label:24} {per_seq/2**20:7.1f} MiB/seq {per_seq*BATCH/2**30:5.2f} GiB at batch {BATCH}")

print(f"state values per seq {values:,} ({linear} linear-attention layers)")
print(f"storage reduction {100*(1-9.9/32):.1f}% <- the paper's headline number, from its own bit budget")
```

FP32, as stored today	108.0 MiB/seq	6.75 GiB at batch 64
DAMP, 9.9 bit/value	33.4 MiB/seq	2.09 GiB at batch 64
state values per seq	28,311,552	(36 linear-attention layers)
storage reduction	69.1%	<- the paper's headline number, from its own bit budget

WATCH OUT

CAVEAT

The paper measures Qwen3.6-35B and Kimi-Linear-48B, not this config. Our numbers apply its stated bit budget to the Flash-Next config from item 01: a projection, not their measurement.

SOURCES & LINKS — every claim above traces to one of these

TYPE	WHAT	URL
Primary source	Original announcement / docs	arxiv.org/abs/2608.27513
Runnable example	This exact script, re-run by CI	github.com/marcocm28/ai-daily-diff/tree/main/examples/2026-09-01-damp-recurrent-state
Slides, video, brief	Everything from this episode	marcocm28.github.io/ai-daily-diff/2026-09-01/

A checkpoint can pass every FP16 check and behave differently after INT8. Certification does not survive quantization.

A paper formalizes quantization as a many-to-one map, then embeds payloads that stay dormant at FP16 and activate after compression.

CLAIMS & RISKS

ARXIV

EXAMPLE — CI VERIFICATION PENDING

THE DIFF

- certify the FP16 checkpoint, ship the INT8 build
- + certify the configuration you actually deploy

WHY IT MATTERS

If you validate the FP16 checkpoint and ship the quantized one, you certified a model you did not deploy.

WHO SHOULD CARE

Anyone shipping quantized weights to edge devices, or accepting third-party checkpoints on trust.

THE NUMBER

0% →
85.02%

friend-foe inversion in their backdoored translation model, after quantization

- A paired stance classifier shifted by up to $\Delta_{\text{bias}} = 0.33$ on compression.
- Persistence varies by quantization scheme, not by nominal bit-width alone.
- Our example is the mechanism at toy scale, not their attack.
- In it: 2232 of 4096 INT8 codes change while weights move 0.4% of $\max|w|$, and the full-precision output on the validation input does not move at all.

RUN IT — 17 lines, numpy, CPU, offline, < 1 s

```
import numpy as np
rng = np.random.default_rng(0)

def int8(w):
    s = np.abs(w).max() / 127
    return np rint(w / s).astype(np.int8), s

w = rng.normal(0, 0.05, 4096).astype(np.float64) # one weight row
x = rng.normal(0, 1.00, 4096).astype(np.float64) # the validation input
q, s = int8(w)

r = w / s - np rint(w / s) # distance to nearest INT8 code
push = np.sign(r) * (0.5 - np.abs(r) + 1e-3) * s # nudge every weight just past its boundary
push -= (push @ x) / (x @ x) * x # ...orthogonal to x: FP64 output unchanged
w_adv = w + push
q_adv, s_adv = int8(w_adv)

print(f"weights moved by {100*np.abs(push).max()/np.abs(w).max():.3f}% of max|w|")
print(f"full precision w@x {w @ x:+.6f} -> {w_adv @ x:+.6f} (identical to 6 dp)")
print(f"after INT8 w@x {(q*s) @ x:+.6f} -> {(q_adv*s_adv) @ x:+.6f}")
print(f"INT8 codes changed {(q != q_adv).sum()}/{q.size}")
```

```
weights moved by 0.406% of max|w|
full precision w@x -0.906887 -> -0.906887 (identical to 6 dp)
after INT8 w@x -0.936712 -> -3.157366
INT8 codes changed 2232/4096
```

WATCH OUT

CAVEAT

Our example demonstrates the structural gap — a perturbation orthogonal to the validation input flipping INT8 codes — not the paper's fine-tuning attack. It is not a reproduction of their 85.02%.

SOURCES & LINKS — every claim above traces to one of these

TYPE	WHAT	URL
Primary source	Original announcement / docs	arxiv.org/abs/2608.27512
Runnable example	This exact script, re-run by CI	github.com/marcocm28/ai-daily-diff/tree/main/examples/2026-09-01-quantization-gap
Slides, video, brief	Everything from this episode	marcocm28.github.io/ai-daily-diff/2026-09-01/